

Lecture 02: Java Syntax Basics

Introduction to Programming with Java

Today's learning goals

By the end of this lecture you should be able to:

1. Declare and initialise variables with appropriate primitive types
2. Write `if/else` and `switch` decisions
3. Use `for`, `while`, and `do-while` loops
4. Create and index arrays; use basic `String` operations
5. Read short programs and predict their output

Agenda

1. Variables and assignment
2. Primitive data types (focus on integers and friends)
3. Conditionals: `if/else` and relational operators
4. `switch`
5. Loops: `for`, `while`, `do-while`
6. Arrays
7. String basics
8. Putting it together

What is a variable?

A **variable** is a named storage location for a value of a given type.

- Has a **name** (identifier)
- Has a **type** (what kind of data it holds)
- Holds a **value** that can change over time (unless `final`)

Declaring and initialising

```
1 int score;           // declaration
2 score = 0;           // assignment
3
4 int lives = 3;        // declaration + initialisation
5
6 final double PI = 3.14159; // constant (cannot reassign)
```

- Java is **statically typed**: the type is fixed at compile time
- Local variables must be assigned before use

Naming rules and style

- Letters, digits, `_`, `$`; cannot start with a digit
- Case-sensitive: `score` $\neq$ `Score`
- Convention: **camelCase** for variables and methods (`playerScore`)
- Class names use **PascalCase** (`PlayerScore`)
- Prefer meaningful names over abbreviations

Common mistake

Using reserved words as names (`int`, `class`, `for`, ...).

Assignment and expressions

```
1 int a = 10;  
2 int b = 3;  
3 int sum = a + b;           // 13  
4 int rem = a % b;           // 1 (remainder)  
5 a += 5;                     // same as a = a + 5  
6 a++;                         // increment by 1
```

Operators you will use constantly: + - * / % and compound assignments += -= *= /=.

Integer division (watch out)

- Dividing two `int` values truncates toward zero
- `7 / 2` is 3, not 3.5
- Use a floating type if you need a fractional result

Try this

Predict `5 / 2`, `5 % 2`, and `5.0 / 2`. Then verify in a tiny program.

Primitive types overview

Java's eight primitive types:

Category	Types	Examples
Integer	byte, short, int, long	42, 100L
Floating	float, double	3.14, 2.5f
Character	char	'A'
Boolean	boolean	true, false

Default choices for beginners: int, double, boolean, char.

Integers in focus

- `int` — 32-bit signed integer; everyday counting and indices
- `long` — 64-bit; use for larger ranges (suffix `L`)
- `byte` / `short` — smaller ranges; less common early on

Ranges (approximate mental model)

`int` roughly ± 2 billion; `long` far larger.

Overflow wraps around—do not rely on silent wrap in logic.

Other primitives you will meet

```
1 double price = 19.99;  
2 boolean isReady = true;  
3 char grade = 'B';  
4  
5 // float is less common in coursework; prefer double  
6 float ratio = 0.5f;
```

- boolean is only true or false (not 0/1)
- char holds a single UTF-16 code unit; use single quotes
- String is *not* a primitive (next sections)

Type casting (gentle intro)

```
1 int whole = 5;
2 double real = whole;           // widening: OK, automatic
3
4 double x = 3.9;
5 int truncated = (int) x;       // narrowing: 3 (fraction dropped)
```

Common mistake

Expecting (int) 3.9 to round to 4. It truncates toward zero.

Quiz 1 — Types and division

Quiz

What does `System.out.println(7 / 2);` print?

1. 3.5
2. 3
3. 4
4. A compile error

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 1

Answer

2. 3 — integer division truncates toward zero.

Use a floating type (e.g. `7.0 / 2` or `(double) 7 / 2`) for a fractional result.

Making decisions

Programs branch based on conditions:

- Relational operators: `==` `!=` `<` `>` `<=` `>=`
- Logical operators: `&&` (and), `||` (or), `!` (not)
- Result of a comparison is a `boolean`

if and else

```
1 int temperature = 18;
2
3 if (temperature < 0) {
4     System.out.println("Freezing");
5 } else if (temperature < 15) {
6     System.out.println("Cold");
7 } else {
8     System.out.println("Mild or warm");
9 }
```

Braces and structure

- Always use braces `{ }` even for one statement—avoids bugs
- Conditions go in parentheses: `if (score >= 50)`
- `else if` chains test alternatives in order
- First matching branch runs; then the chain ends

Common mistake

Writing `if (x = 5)` instead of `if (x == 5)`. Assignment vs equality—Java often catches this because types must be boolean.

Logical operators

```
1 int age = 20;
2 boolean hasId = true;
3
4 if (age >= 18 && hasId) {
5     System.out.println("Entry allowed");
6 }
7
8 if (age < 12 || age > 65) {
9     System.out.println("Discount eligible");
10 }
11
12 if (!hasId) {
13     System.out.println("Please show ID");
14 }
```

Short-circuit evaluation

- `&&` stops if the left side is `false`
- `||` stops if the left side is `true`
- Useful and important when later conditions would be invalid

Example idea: check array length before indexing (arrays come soon).

switch: multi-way branch

```
1 int day = 3;
2 switch (day) {
3     case 1:
4         System.out.println("Monday");
5         break;
6     case 2:
7         System.out.println("Tuesday");
8         break;
9     case 3:
10        System.out.println("Wednesday");
11        break;
12    default:
13        System.out.println("Other day");
14 }
```

switch rules of thumb

- Classic `switch` works well with `int`, `char`, `String`, `enums`
- Include `break` unless you intentionally want fall-through
- `default` handles unmatched values

Common mistake

Forgetting `break` and “falling through” into the next case.

Modern switch (optional awareness)

Java also supports a clearer arrow form (Java 14+):

```
1 int day = 3;
2 String name = switch (day) {
3     case 1 -> "Monday";
4     case 2 -> "Tuesday";
5     case 3 -> "Wednesday";
6     default -> "Other day";
7 };
8 System.out.println(name);
```

Use what your course JDK supports; classic `switch` is still essential.

Quiz 2 — What does this print?

```
1 int x = 10;  
2 if (x > 15) {  
3     System.out.println("A");  
4 } else if (x > 5) {  
5     System.out.println("B");  
6 } else {  
7     System.out.println("C");  
8 }
```

Quiz

Output?

1. A
2. B
3. C
4. Nothing (no branch runs)

Answer

2. B — $x > 15$ is false; $x > 5$ is true, so that branch runs and the chain stops.

Quiz 2b — switch (true/false)

Quiz

True or false?

In a classic switch, forgetting break can cause execution to continue into the next case (fall-through).

1. True
2. False

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

True. Always add `break` unless you intentionally want fall-through.

Why loops?

Repeat work without copying code:

- Process every element of a collection
- Repeat until a condition becomes false
- Count from a start value to an end value

Three core forms: `for`, `while`, `do-while`.

for loop

```
1 for (int i = 0; i < 5; i++) {  
2     System.out.println(i);  
3 }  
4 // prints 0 1 2 3 4 (each on its own line)
```

Anatomy: **initialisation**; **condition**; **update**.

- Prefer starting at 0 when working with arrays later
- Condition checked *before* each iteration

while loop

```
1 int n = 3;  
2 while (n > 0) {  
3     System.out.println(n);  
4     n--;  
5 }  
6 System.out.println("Go!");
```

Use `while` when the number of iterations is not obvious in advance.

Common mistake

Forgetting to update the loop variable → infinite loop.

do-while loop

```
1 int guess;  
2 do {  
3     guess = readGuess();    // pretend this reads input  
4 } while (guess < 1 || guess > 10);
```

- Body runs **at least once**
- Condition checked *after* the body
- Handy for “ask until valid” patterns

Choosing a loop

Prefer	When
<code>for</code>	Counting; known range; array indices
<code>while</code>	Repeat while a condition holds
<code>do-while</code>	Must run once; then maybe again

Try this

Rewrite a `for` that prints 1..10 as a `while`. Same output?

break and continue (brief)

```
1 for (int i = 1; i <= 10; i++) {  
2     if (i == 3) {  
3         continue;    // skip the rest of this iteration  
4     }  
5     if (i == 8) {  
6         break;        // leave the loop entirely  
7     }  
8     System.out.println(i);  
9 }
```

Use sparingly; clear loop conditions are often easier to read.

Quiz 3 — Loop output

```
1 int n = 3;  
2 while (n > 0) {  
3     System.out.print(n);  
4     n--;  
5 }
```

Quiz

What is printed?

1. 321
2. 333
3. 012
4. Nothing (infinite loop, never prints)

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 3

Answer

1. 321 — prints while `n` is 3, then 2, then 1; then `n` becomes 0 and the loop ends.

What is an array?

An **array** stores a fixed number of values of the *same* type, accessed by an integer index.

- Length is fixed after creation
- Indices start at 0
- Last index is `length - 1`

Creating and using arrays

```
1 int[] scores = new int[3];    // {0, 0, 0}
2 scores[0] = 90;
3 scores[1] = 85;
4 scores[2] = 92;
5
6 int[] primes = {2, 3, 5, 7};
7 System.out.println(primes.length);    // 4
8 System.out.println(primes[0]);        // 2
```

Traversing an array

```
1 int[] values = {4, 1, 9, 2};
2 int sum = 0;
3
4 for (int i = 0; i < values.length; i++) {
5     sum += values[i];
6 }
7 System.out.println(sum);    // 16
8
9 // enhanced for (read each element):
10 for (int v : values) {
11     System.out.println(v);
12 }
```

Bounds and defaults

- Accessing `values[values.length]` throws `ArrayIndexOutOfBoundsException`
- Numeric arrays default to 0; boolean to false; reference types to null

Common mistake

Using `<= values.length` in a for loop header. Use `< values.length`.

Arrays of other types

```
1 double[] temps = {18.5, 21.0, 19.2};  
2 boolean[] flags = {true, false, true};  
3 char[] letters = {'J', 'a', 'v', 'a'};  
4 String[] names = {"Ada", "Alan", "Grace"};
```

Multidimensional arrays exist (`int[][]`), but master 1D arrays first.

Quiz 4 — Arrays

```
1 int[] a = {10, 20, 30};  
2 System.out.println(a.length);  
3 System.out.println(a[0]);
```

Quiz

What is printed (two lines)?

1. 2 then 10
2. 3 then 10
3. 3 then 20
4. 3 then a compile error

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 4

Answer

2. 3 then 10 — length is 3; index 0 is the first element.

Last valid index is `a.length - 1` (here: 2).

String is a class

- `String` holds text: sequences of characters
- Not a primitive—it is a reference type (objects)
- Literals use double quotes: `"Hello"`
- Strings are **immutable**: operations create new strings

Creating and concatenating

```
1 String first = "Hello";
2 String second = "Java";
3 String msg = first + ", " + second + "!";
4 System.out.println(msg);           // Hello, Java!
5
6 int n = 7;
7 System.out.println("Count: " + n); // Count: 7
```

Useful String methods

```
1 String s = "Java";  
2 System.out.println(s.length());           // 4  
3 System.out.println(s.charAt(0));           // J  
4 System.out.println(s.toUpperCase());       // JAVA  
5 System.out.println(s.substring(1, 3));     // av  
6  
7 String t = "java";  
8 System.out.println(s.equals(t));           // false  
9 System.out.println(s.equalsIgnoreCase(t)); // true
```

Comparing strings correctly

```
1 String a = "hi";
2 String b = "hi";
3 String c = new String("hi");
4
5 // Prefer equals for content:
6 System.out.println(a.equals(b)); // true
7 System.out.println(a.equals(c)); // true
8
9 // == compares references (identity), not always content
```

Common mistake

Using `==` to compare String contents. Use `equals` (or `equalsIgnoreCase`).

char vs String

	char	String
Quotes	single 'A'	double "A"
Meaning	one character	text (any length)
Kind	primitive	object

Quiz 5 — Comparing strings

Quiz

Which is the recommended way to compare the *text content* of two `String` values `s` and `t`?

1. `s == t`
2. `s.equals(t)`
3. `s = t`
4. `s.compare(t)`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. `s.equals(t)` (or `equalsIgnoreCase` when case should not matter).

`==` compares references (identity), which is not reliable for string *content*.

Mini example: average of scores

```
1 int[] scores = {80, 90, 70, 100};
2 int sum = 0;
3 for (int i = 0; i < scores.length; i++) {
4     sum += scores[i];
5 }
6 double average = (double) sum / scores.length;
7 System.out.println("Average: " + average);
8
9 if (average >= 90) {
10     System.out.println("Excellent");
11 } else if (average >= 70) {
12     System.out.println("Good");
13 } else {
14     System.out.println("Keep practising");
15 }
```

Mini example: count vowels

```
1 String word = "Education";
2 int vowels = 0;
3 String lower = word.toLowerCase();
4
5 for (int i = 0; i < lower.length(); i++) {
6     char c = lower.charAt(i);
7     if (c == 'a' || c == 'e' || c == 'i'
8         || c == 'o' || c == 'u') {
9         vowels++;
10    }
11 }
12 System.out.println("Vowels: " + vowels);
```

Debugging checklist

When something fails:

1. Read the compiler error: line number + message
2. Check types and missing semicolons / braces
3. Trace loop bounds and array indices
4. Print intermediate values (`System.out.println`)
5. Change one thing; recompile; rerun

Key takeaways

1. Variables need a type, a name, and a defined value before use
2. Prefer `int`, `double`, `boolean`, `char` early on
3. `if/else` and `switch` express decisions
4. `for/while/do-while` express repetition
5. Arrays: fixed length, 0-based indices
6. Strings: use methods and `equals`, not `==` for content

Practice before next time

Try this

Write a program that reads (or hard-codes) five integers into an array, prints the maximum, and prints whether the maximum is even or odd.

Coming next

Methods, parameters, and structuring larger programs— building on today's control flow and data basics.

Which construct do you want to see one more example of?